

ODOO DEVELOPER PROGRAM

ORM Methods

Odoo 17 Development Tutorials

Ahmed Muhumed

July 25, 2026

- 1 Introduction to ORM Methods
- 2 `Create()` ORM Method
- 3 `write()` ORM Method

Introduction to ORM Methods

WHAT IS ORM?

- ▶ The ORM stands for **Object Relational Mapping**.
- ▶ It is a programming technique that allows developers to interact with a database using **Python objects and methods** instead of writing **SQL queries directly**.
- ▶ In Odoo, the ORM acts as a **bridge** between the Python application and the PostgreSQL database.
- ▶ Instead of writing:

```
1 INSERT INTO student_student (name, age) VALUES ( 'Ahmed' , 22);
```

- ▶ we simply write:

```
1 self.env[ 'student.student' ].create({  
2     'name': 'Ahmed',  
3     'age': 22,  
4 })
```

WHAT DOES "OBJECT RELATIONAL MAPPING" MEAN?

- ▶ The term consists of three parts.

1 **Object** - an object is an instance of a Python class.

```
1 student = self.env['student.student'].browse(1)
```

- Here, **student** is a Python object representing a student.

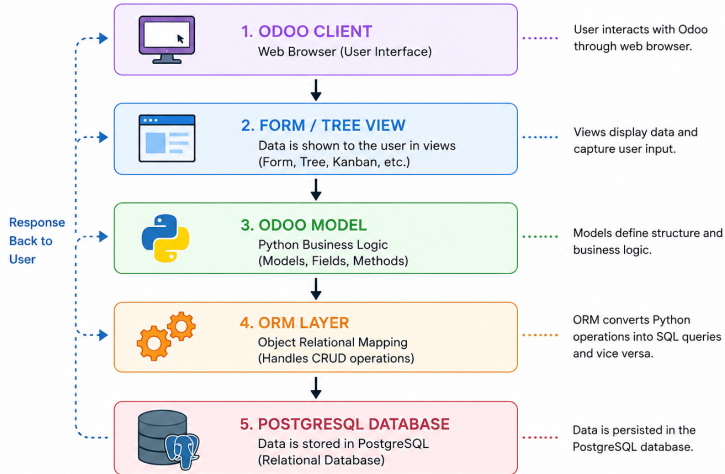
2 **Relational** - a relational database stores data in tables that are connected using relationships.

Students	Classes
Mohamed	Grade 12

3 **Mapping** - mapping means connecting two different worlds.

Python Object $\Rightarrow$ Odoo ORM $\Rightarrow$ PostgreSQL Table

ORM Architecture in Odoo



WHY IS ORM IMPORTANT?

- ▶ Without an ORM, developers would need to:
 - Write SQL for every operation.
 - Manage database connections manually.
 - Convert database rows into Python objects.
 - Prevent SQL injection.
- ▶ Although powerful, the ORM has limitations.
 - Less control over complex SQL queries.
 - Very large reports may require raw SQL.
 - Advanced database tuning sometimes needs direct SQL.

COMMON ORM METHODS

- ▶ create
- ▶ write
- ▶ unlink
- ▶ copy
- ▶ exists
- ▶ search
- ▶ read
- ▶ search_read
- ▶ search_count
- ▶ name_create
- ▶ default_get
- ▶ _name_search
- ▶ get_view()
- ▶ ensure_one()
- ▶ filtered()
- ▶ mapped()
- ▶ sorted()
- ▶ grouped()
- ▶ fields_get()
- ▶ get_metadata

Create () ORM Method

CREATE () METHOD

- ▶ The `create()` method is an ORM (Object Relational Mapping) method used to create one or more new records in an Odoo model.
- ▶ The `create()` method returns a **recordset** containing the newly created record.

Method Syntax

- The standard syntax of the `create()` method is:

```
1  @api.model
2  def create(self, vals):
3      return super().create(vals)
4
```

- This method has four important parts:

- 1 `@api.model`
- 2 `create`
- 3 `self`
- 4 `vals`

BREAKING DOWN THE METHOD SIGNATURE

- ▶ **@api.model**: Indicates that the method is executed at the model level. The `@api.model` decorator tells Odoo that the method belongs to the model, not to an existing record.
- ▶ **create**: The ORM method responsible for creating records.
- ▶ **self**: The model (`student.student`), not a specific student record. If we are using `student.student` model, `self` does not represent a student record. Instead, `self`, represents the model `student.student`.

```
1 print(self)
```

Output is:

```
1 student.student()
```

BREAKING DOWN THE METHOD SIGNATURE

- **vals**: A dictionary containing the values to be stored in the database.

```
1      name = fields.Char()  
2      birth_date = fields.Date()  
3      gender = fields.Selection(...)
```

```
1      vals = {  
2          'name': 'Ahmed Mohamed',  
3          'birth_date': date(2005, 5, 10),  
4          'gender': 'male',  
5      }
```

- Each **key** corresponds to a field name in your model, and
- Each **value** is the data to store.
- The ORM maps each key to its corresponding field.

WHAT DOES `CREATE()` RETURN?

- ▶ The **`create()`** method returns a recordset.

```
1  student = self.env['student.student'].create({  
2      'name': 'Ahmed',  
3      })
```

- ▶ Now, if we try to print the **`student`** using **`print(student)`**, we are getting this

```
1  student.student(15,)  
2
```

@API.MODEL VS @API.MODEL_CREATE_MULTI

There are two common decorators for create().

- 1 **@api.model**: Used for creating a single record.

```
1 @api.model
2 def create(self, vals):
```

■ **vals** is a dictionary.

```
1 student = self.env['student.student'].create({
2     'name': 'Ahmed Mohamed',
3     })
```

- 2 **@api.model_create_multi**: Used for creating multiple records efficiently.

```
1 @api.model_create_multi
2 def create(self, vals_list):
```

@API.MODEL VS @API.MODEL_CREATE_MULTI

- ▶ **vals_list** is a list of dictionaries, i.e.,

```
1  [  
2    { 'name': 'Ahmed' } ,  
3    { 'name': 'Amina' } ,  
4  ]
```

Method Call

- ▶ A method call is when we **execute** the method.

```
1  student = self.env['student.student'].create({  
2      'name': 'Ahmed Mohamed',  
3      'email': 'ahmed@example.com',  
4  })  
5
```

- ▶ Here, **.create(...)** is the **method call**.

`write()` ORM Method

